GAN , Generative Adversarial Networks

모두의 딥러닝 개정4판

생성형 적대적 신경망

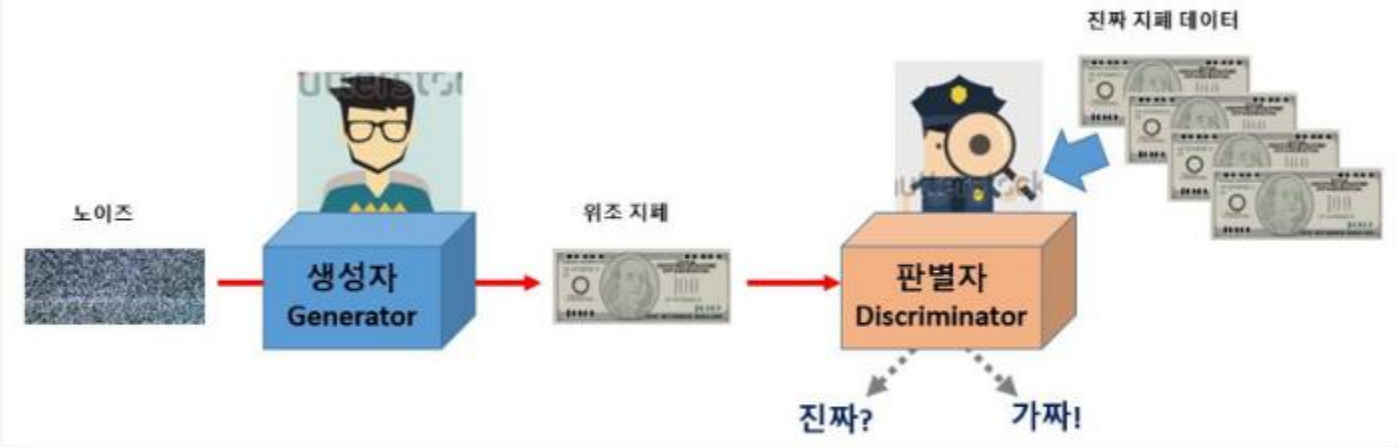
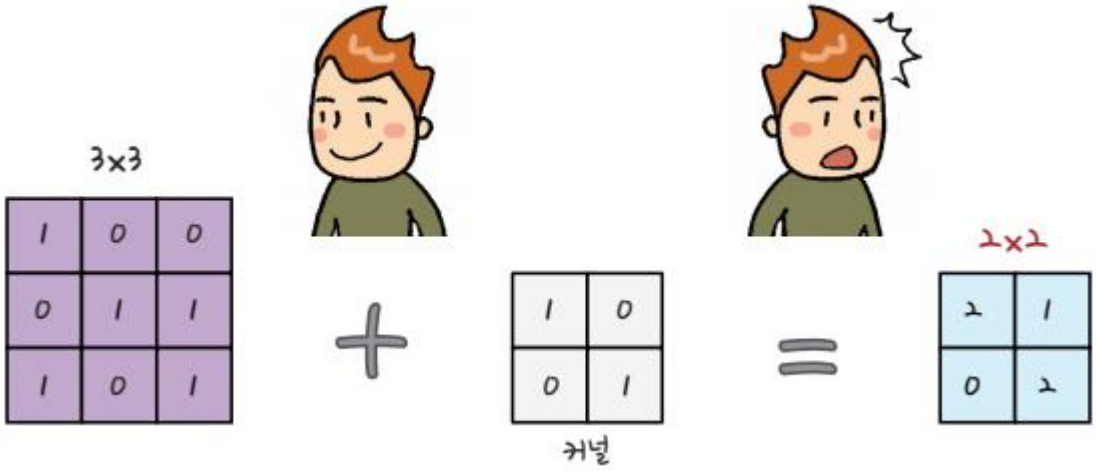


Figure 1. 생성자는 판별자가 구별할 수 없을 만큼 정교한 가짜를 만드는게 목표!

1 가짜 제조 공장, 생성자

- **가짜 제조 공장, 생성자 (페이스북 연구팀 Deep Convolutional GAN)**
  - **생성자**(generator)는 가상의 이미지를 만들어 내는 공장
  - DCGAN은 생성자가 가짜 이미지를 만들 때 컨볼루션 신경망(CNN)을 이용한다고 했음
  - 먼저 옵티마이저(optimizer)를 사용하는 최적화 과정이나 컴파일하는 과정이 없다는 것
  - 판별과 학습이 이곳 생성자에서 일어나는 것이 아니기 때문임
  - 일부 매개변수를 삭제하는 풀링(pooling) 과정이 없는 대신 패딩(padding) 과정이 포함

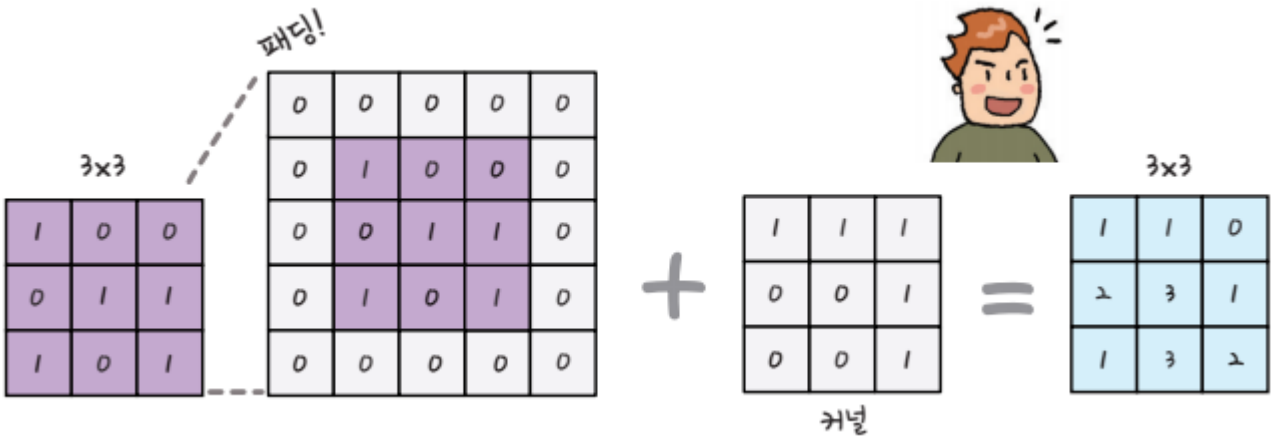
▼ 그림 17-2 | 생성자의 결과물이 실제 데이터와 같은 크기여야 하므로 차원 축소 문제를 해결해야 한다



1 가짜 제조 공장, 생성자

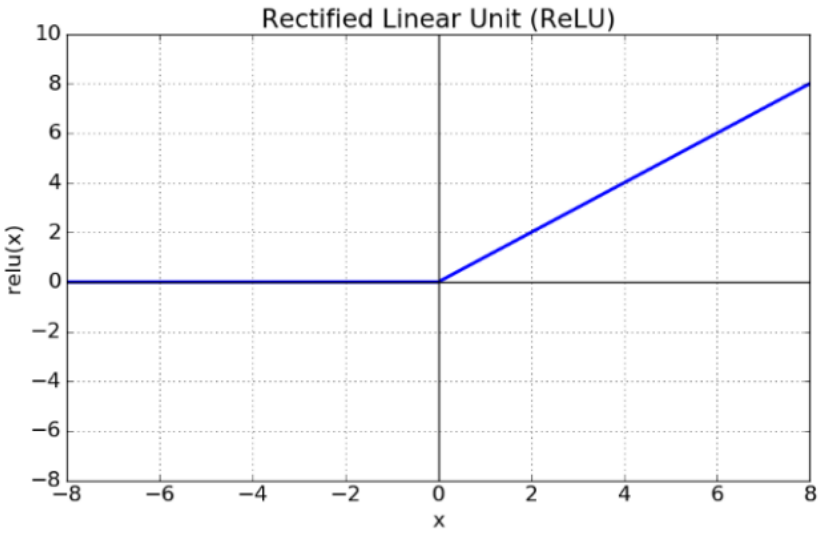
- **가짜 제조 공장, 생성자**
  - 패딩 과정을 통해 생성자가 만들어 내는 이미지의 크기를 조절해야 하는 이유는 판별자가 비교할 '진짜'와 똑같은 크기가 되어야 하기 때문임
  - 케라스의 패딩 함수는 이러한 문제를 쉽게 처리할 수 있도록 도와줌
  - padding='same'이라는 설정을 통해 입력과 출력의 크기가 다를 경우 자동으로 크기를 확장해 주고, 확장된 공간에 0을 채워 넣을 수 있음

▼ 그림 17-3 | 패딩을 이용하면 커널을 거친 후에도 차원의 변동이 없게 만들 수 있다!

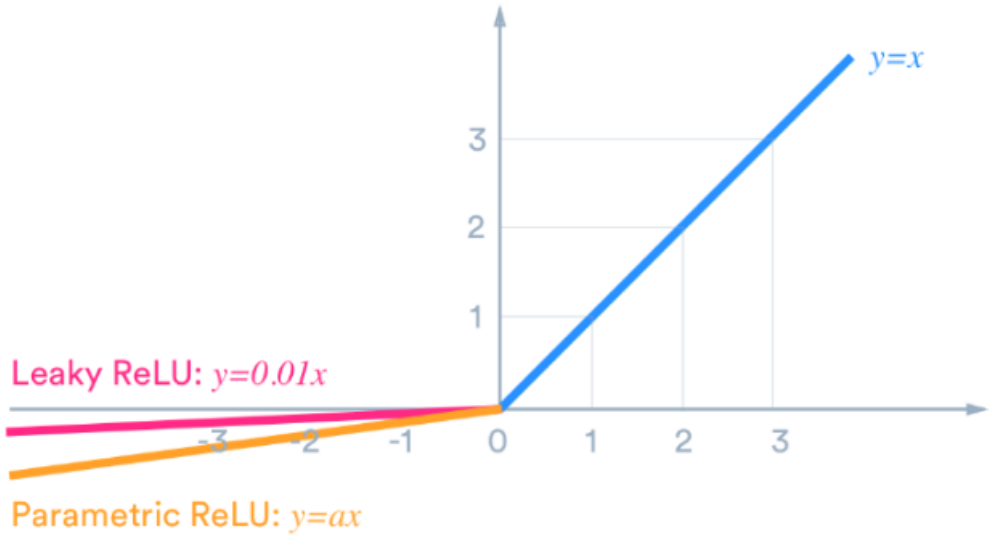


1 가짜 제조 공장, 생성자

- ❶과 ❼ 에서 활성화 함수로 LeakyReLU(리키렐루)를 썼음 (0보다 작을 경우 0.2를 곱함)
- GAN에서는 기존에 사용하던 ReLU() 함수를 쓸 경우 학습이 불안정해지는 경우가 많아, ReLU()를 조금 변형한 LeakyReLU() 함수를 씀



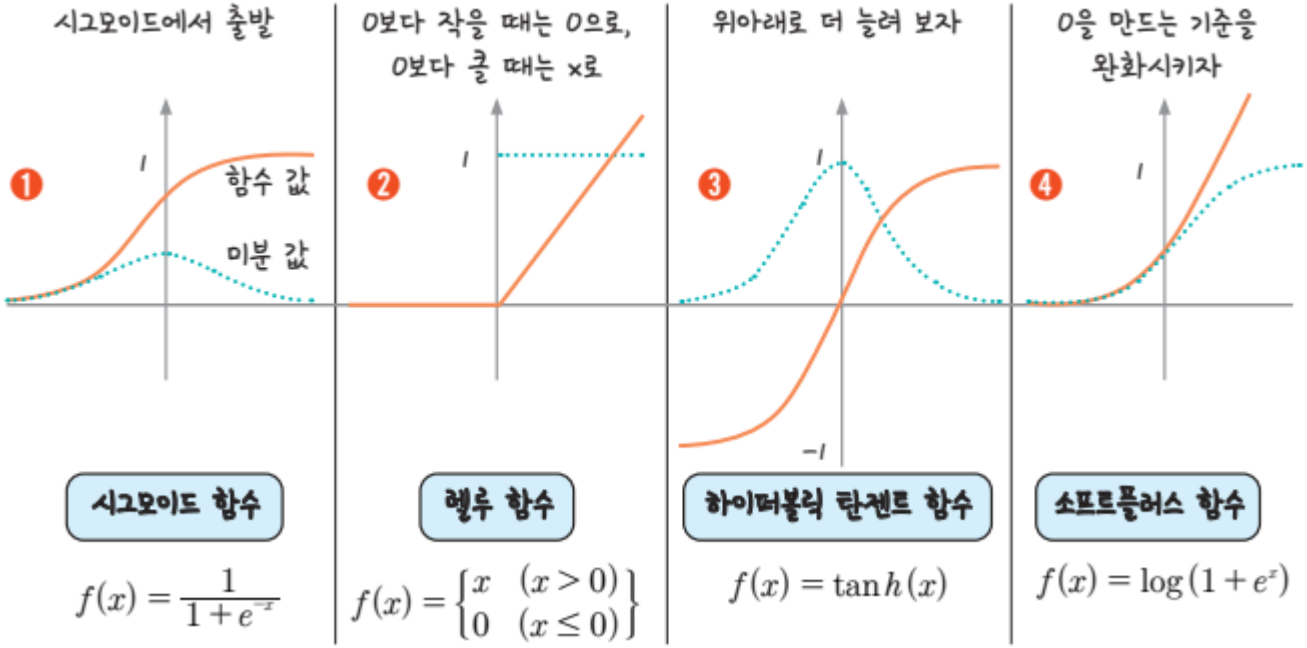
④ Leaky ReLU, Parametric ReLU 함수



1 가짜 제조 공장, 생성자

- **가짜 제조 공장, 생성자**
  - 끝으로 ❾에서 한 번 더 컨볼루션 과정을 거친 후 판별자로 값을 넘길 준비를 마침
  - 활성화 함수는 tanh() 함수를 썼음

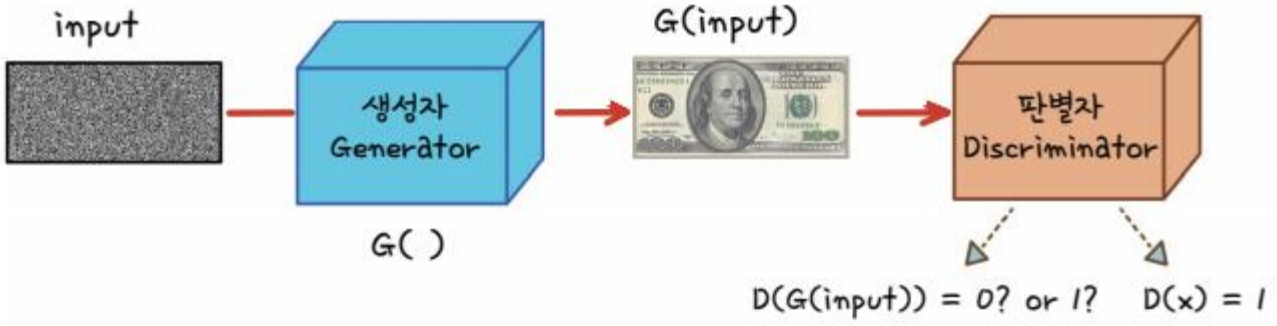
```
generator.add(Conv2D(1, kernel_size=5, padding='same', activation='tanh'))
..... ❾
```



3 적대적 신경망 실행하기

- **적대적 신경망 실행하기**
  - 이제 생성자와 판별자를 연결시키고 학습을 진행하며 기타 여러 가지 옵션을 설정하는 순서
  - 생성자와 판별자를 연결시킨다는 것은 생성자에서 나온 출력을 판별자에 넣어서 진위 여부를 판별하게 만든다는 의미

▼ **그림 17-4 | 생성자 모델이 G(), 판별자 모델이 D(), 실제 데이터가 x, 입력 값이 input일 때 GAN의 기본 구조**



3 적대적 신경망 실행하기

discriminator.train_on_batch(imgs, true) 코드에서 반환되는 d_loss_real은 "판별자(Discriminator)가 진짜 이미지(imgs)를 받았을 때, 이를 얼마나 '진짜'라고 잘 맞추었는가"를 평가한 손실(Loss) 값입니다.

discriminator.train_on_batch(gen_imgs, fake) 코드에서 반환되는 d_loss_fake는 "판별자(Discriminator)가 생성자가 만든 가짜 이미지(gen_imgs)를 받았을 때, 이를 얼마나 '가짜'라고 잘 잡아냈는가"를 평가한 손실(Loss) 값입니다.

- d_loss가 작다는 것은 일반적으로 판별자가 실제 데이터와 가짜 데이터를 잘 구분한다는 의미입니다. (판별자 가중치 업데이트)
- g_loss가 작을수록 생성자가 판별자를 더 잘 속이고 있다는 의미입니다 (생성자 가중치 업데이트)
- 이론적으로 이상적인 균형에서는 D(G(z))=0.5이며, 표준 BCE(Binary Cross Entropy)를 사용하면 두 손실은 약 0.693이 됩니다.
- 다만 d_loss와 g_loss가 모두 0에 가까워지는 것이 GAN의 최종 목표는 아닙니다. 두 모델이 균형 있게 경쟁하면서 생성 데이터의 품질이 향상되는 것이 중요합니다.

3 적대적 신경망 실행하기


```
# d_loss_real, d_loss_fake 값을 더해 둘로 나눈 평균이 바로 판별자의 오차
d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)
```

```
g_loss = gan.train_on_batch(noise, true)
```

- d_loss가 작다는 것은 일반적으로 판별자가 실제 데이터와 가짜 데이터를 잘 구분한다는 의미입니다. (판별자 가중치 업데이트)
- g_loss가 작을수록 생성자가 판별자를 더 잘 속이고 있다는 의미입니다 (생성자 가중치 업데이트)
- 이론적으로 이상적인 균형에서는 D(G(z))=0.5이며, 표준 BCE(Binary Cross Entropy)를 사용하면 두 손실은 약 0.693이 됩니다.
- 다만 d_loss와 g_loss가 모두 0에 가까워지는 것이 GAN의 최종 목표는 아닙니다. 두 모델이 균형 있게 경쟁하면서 생성 데이터의 품질이 향상되는 것이 중요합니다.

4 이미지의 특징을 추출하는 오토인코더

- **이미지의 특징을 추출하는 오토인코더**
  - 오토인코더의 학습은 GAN의 학습보다 훨씬 쉬움

▼ **그림 17-7 | 오토인코더의 구조. 입력 값(X)과 출력 값(X') 사이에 차원이 작은 노드가 포함됨**

